Go Modules

What is a Go Module?

A Go module is a collection of Go packages with a go.mod file at its root. It defines the module's name, Go version, and dependencies.

Step 1: Initialize a New Module

1. Action: Create a new directory and run the initialization command.
2. Command: go mod init <module-name>

Usually, your module name is its repository path (e.g., github.com/username/my-project). If you're just playing around locally, example.com/hello works fine.

mkdir myproject

cd myproject

go mod init github.com/yourname/myproject

Adding Dependencies

When you import a third-party package in your code and run a command like go build or go test, Go automatically fetches the dependency for you.

- **Manual Fetching:** You can also explicitly add a package using: go get github.com/google/uuid
- **What happens:** This updates your go.mod file and creates a go.sum file.

Custom Modules

Create a Go module

Go code is grouped into packages, and packages are grouped into modules

1. Create a module -- Write a small module with functions you can call from another module.
2. Call your code from another module -- Import and use your new module.

Open a command prompt and cd to your home directory.

On Windows:

cd %HOMEPATH%

Create a greetings directory for your Go module source code.

For example, from your home directory use the following commands:

mkdir greetings

cd greetings

Start your module using the go mod init command.

Run the go mod init command, giving it your module path -- here, use example.com/greetings. If you publish a module, this *must* be a path from which your module can be downloaded by Go tools. That would be your code's repository.

$ go mod init example.com/greetings

go: creating new go.mod: module example.com/greetings

The go mod init command creates a go.mod file to track your code's dependencies. So far, the file includes only the name of your module and the Go version your code supports. But as you add dependencies, the go.mod file will list the versions your code depends on

In your text editor, create a file in which to write your code and call it greetings.go.

Paste the following code into your greetings.go file and save the file

```
package greetings

import "fmt"

// Hello returns a greeting for the named person.
func Hello(name string) string {
   // Return a greeting that embeds the name in a message.
   message := fmt.Sprintf("Hi, %v. Welcome!", name)
```

```
    return message
}
```

This is the first code for your module. It returns a greeting to any caller that asks for one. You'll write code that calls this function in the next step.

In this code, you:

- Declare a greetings package to collect related functions.
- Implement a Hello function to return the greeting

Call your code from another module

In the previous section, you created a greetings module. In this section, you'll write code to make calls to the Hello function in the module you just wrote

Create a hello directory for your Go module source code. This is where you'll write your caller.

After you create this directory, you should have both a hello and a greetings directory at the same level in the hierarchy, like so:

```
<home>/
        |-- greetings/
        |-- hello/
```

Enable dependency tracking for the code you're about to write.

To enable dependency tracking for your code, run the go mod init command, giving it the name of the module your code will be in.

For the purposes of this tutorial, use example.com/hello for the module path.

```
$ go mod init example.com/hello
```

```
go: creating new go.mod: module example.com/hello
```

In your text editor, in the hello directory, create a file in which to write your code and call it hello.go.

Write code to call the Hello function, then print the function's return value

To do that, paste the following code into hello.go.

```
package main

import (
   "fmt"

   "example.com/greetings"
)

func main() {
   // Get a greeting message and print it.
   message := greetings.Hello("Gladys")
   fmt.Println(message)
}
```

Edit the example.com/hello module to use your local example.com/greetings module

For production use, you'd publish the example.com/greetings module from its repository (with a module path that reflected its published location), where Go tools could find it to download it.

For now, because you haven't published the module yet, you need to adapt the example.com/hello module so it can find the example.com/greetings code on your local file system

To do that, use the go mod edit command to edit the example.com/hello module to redirect Go tools from its module path (where the module isn't) to the local directory (where it is).

1. From the command prompt in the hello directory, run the following command:
2. $ go mod edit -replace example.com/greetings=../greetings
3. The command specifies that `example.com/greetings` should be replaced with `../greetings` for the purpose of locating the dependency. After you run the command, the go.mod file in the hello directory should include a `replace` directive:

```
module example.com/hello

go 1.16

replace example.com/greetings => ../greetings
```

From the command prompt in the hello directory, run the go mod tidy command to synchronize the example.com/hello module's dependencies, adding those required by the code, but not yet tracked in the module.

$ go mod tidy

go: found example.com/greetings in example.com/greetings v0.0.0-00010101000000-000000000000

After the command completes, the example.com/hello module's go.mod file should look like this:

module example.com/hello

go 1.16

replace example.com/greetings => ../greetings

require example.com/greetings v0.0.0-00010101000000-000000000000

At the command prompt in the hello directory, run your code to confirm that it works.
$ go run .
Hi, Gladys. Welcome!

Understanding the Files

After adding a dependency, you'll see two key files:

File	Purpose
go.mod	The "source of truth." It lists the module name, the Go version, and required dependencies.
go.sum	A list of checksums. It ensures that the dependencies you downloaded today are the exact same ones your teammate downloads tomorrow. **Never edit this manually.**

Cleaning Up (The "Tidy" Command)

As you code, you might add imports and then delete them. Your go.mod file can get cluttered with "ghost" dependencies you no longer use.

- **Command:** go mod tidy
- **Result:** This removes unused modules and adds missing ones. It is the "gold standard" for keeping your project lean.

Upgrading or Downgrading

Need a newer version of a library? Or perhaps a specific older version because the new one broke your build?

- **Upgrade to latest:** go get github.com/user/repo@latest
- **Specific version:** go get github.com/user/repo@v1.2.3

Using "Vendoring" (Optional)

If you want to keep a physical copy of all your dependencies inside your project folder (useful for offline builds or strict enterprise audits), you can use:

- **Command:** go mod vendor
- **Result:** This creates a /vendor directory containing all the source code of your dependencies

Summary of Essential Commands

- go mod init: Start a new module.
- go mod tidy: Clean up unused dependencies.
- go get: Add or update dependencies.
- go list -m all: See all currently used versions.

Directive	Purpose
module	Module's import path
go	Minimum Go version
require	Direct dependencies
replace	Swap a dependency with a local or forked version
exclude	Exclude a specific version

Add Dependencies

bash

```
go get github.com/gin-gonic/gin        # latest version
```

go get github.com/gin-gonic/gin@v1.9.1 # specific version

go get github.com/gin-gonic/gin@latest # explicit latest

This automatically: - Updates `go.mod` with the `require` entry - Creates/updates `go.sum` with cryptographic hashes

`go.sum` ``` github.com/gin-gonic/gin v1.9.1 h1:4idEAncQnU5cB7BeOkPtxjfCSye0AAm1R0RVIqJ+Jmg= github.com/gin-gonic/gin v1.9.1/go.mod h1:hPzmOzneLJ2yl6BBWehJJDMF2ZkHiZOzMwX2i7eoxqI=

1. Never edit this file manually — it's auto-managed
2. Ensures reproducible, tamper-proof builds
3. Commit both go.mod and go.sum to version control

Tidy Dependencies
go mod tidy
This command:
- Adds missing dependencies
- Removes unused dependencies
- Updates both go.mod and go.sum

Run this regularly, especially after refactoring.

Vendor Dependencies (Optional)
bash
```
go mod vendor       # copies all deps into /vendor directory
go build -mod=vendor # build using vendored deps
```

Useful Day-to-Day Commands
bash
```
go list -m all          # list all dependencies
go list -m -u all       # list deps with available updates
go get -u ./...         # upgrade all deps to latest minor/patch
go mod graph            # print dependency graph
go mod verify           # verify deps match go.sum hashes
```

```
go mod why github.com/pkg   # explain why a dep is needed
go mod download             # download deps to local cache
```